```
"""Simple 3-slot JSON save system for campaign progress."""
import json
import os
import time

SAVE_FILE = os.path.join(os.path.abspath("."), "save_data.json")
SLOT_COUNT = 3
_current_slot = 0

def _empty_slot(slot_index):
    return {
        "slot": slot_index,
        "occupied": False,
        "last_completed_mission": "",
        "last_completed_label": "No missions completed",
        "total_time": "00:00",
        "updated_at": 0,
        "stats": {},
        "flags": {},
    }

def _default_data():
    return {"version": 1, "slots": [_empty_slot(i) for i in range(SLOT_COUNT)]}

def load_data():
    if not os.path.exists(SAVE_FILE):
        return _default_data()
    try:
        with open(SAVE_FILE, "r", encoding="utf-8") as f:
            data = json.load(f)
    except Exception:
        return _default_data()
    slots = data.get("slots", [])
    fixed = _default_data()
    for i in range(SLOT_COUNT):
        if i < len(slots) and isinstance(slots[i], dict):
            merged = _empty_slot(i)
            merged.update(slots[i])
            merged["slot"] = i
            fixed["slots"][i] = merged
    return fixed

def save_data(data):
    try:
        with open(SAVE_FILE, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2)
    except Exception as e:
        print(f"Save failed: {e}")

def select_slot(slot_index):
    global _current_slot
    _current_slot = max(0, min(SLOT_COUNT - 1, int(slot_index)))

def current_slot_index():
    return _current_slot

def get_slot(slot_index=None):
    idx = _current_slot if slot_index is None else max(0, min(SLOT_COUNT - 1, int(slot_index)))
    return load_data()["slots"][idx]

def slot_has_save(slot_index=None):
    return bool(get_slot(slot_index).get("occupied"))

def start_new_campaign(slot_index=None):
    idx = _current_slot if slot_index is None else max(0, min(SLOT_COUNT - 1, int(slot_index)))
    data = load_data()
    slot = _empty_slot(idx)
    slot.update({
        "occupied": True,
        "last_completed_mission": "",
        "last_completed_label": "Campaign started - Mission 1 in progress",
        "updated_at": int(time.time()),
        "stats": {},
        "flags": {"campaign_started": True},
    })
    data["slots"][idx] = slot
    save_data(data)

def save_mission_complete(mission_id, label, total_time, stats=None, flags=None, slot_index=None):
    idx = _current_slot if slot_index is None else max(0, min(SLOT_COUNT - 1, int(slot_index)))
    data = load_data()
    slot = _empty_slot(idx)
    slot.update({
        "occupied": True,
        "last_completed_mission": mission_id,
        "last_completed_label": label,
        "total_time": total_time,
        "updated_at": int(time.time()),
        "stats": stats or {},
        "flags": flags or {},
    })
    data["slots"][idx] = slot
    save_data(data)

def format_timestamp(ts):
    if not ts:
        return "Never"
    try:
        return time.strftime("%Y-%m-%d %H:%M", time.localtime(ts))
    except Exception:
        return "Unknown"

def has_any_save():
    return any(bool(slot.get("occupied")) for slot in load_data().get("slots", []))
```